
◆

Arbre de Steiner et Bulles de Savon

Application web interactive de calcul et visualisation

◆

AMEDIAZ Hamid
BENAQA Saad
ALAOUI KASMI Anass

Rapport de projet — Cours *Algorithmes 2*
Polytech Nice Sophia — SI4, Sciences Informatiques
Année universitaire 2025/2026 — 10 avril 2026

Résumé. Réalisation d'une application web interactive résolvant le problème de l'arbre de Steiner euclidien. Backend Java/Spring Boot avec algorithmes exacts pour 2 à 5 points et approximation MST pour $n \geq 6$; frontend Angular 19 pour la visualisation et la comparaison des solutions.

Table des matières

Table des matières

1	Présentation du projet et contexte	3
1.1	Problème posé	3
1.2	Lien avec les bulles de savon	3
1.3	Formulation sur graphe	3
2	Organisation et gestion de projet	3
2.1	Répartition du travail	3
2.2	Rôle de l'intelligence artificielle	4
2.3	Méthodologie	4
3	Partie algorithmique	4
3.1	Vue d'ensemble	4
3.2	Algorithme de Weiszfeld — 3 points	4
3.3	Cas 4 points — 16 topologies	4
3.4	Cas 5 points — ~ 241 topologies	5
3.5	Cas $n \geq 6$ — MST	5
3.6	Validation Python	5
4	Architecture de l'application	6
4.1	Frontend Angular	6
4.2	Backend Java et API REST	6
5	Prise de recul	6
5.1	Bilan	6
5.2	Perspective — Extension à un grand nombre de points	6
6	Références bibliographiques	6

1 Présentation du projet et contexte

1.1 Problème posé

Étant donné un ensemble de **villes dans le plan**, il s'agit de construire un **réseau routier de longueur totale minimale** reliant toutes les villes. On autorise l'ajout de **points de Steiner** — points intermédiaires sans ville correspondante — qui permettent de réduire la longueur totale du réseau.

Propriété fondamentale : dans la solution optimale, chaque point de Steiner est à la jonction d'**exactement trois arêtes formant des angles de 120°** (point de Fermat-Torricelli — Gilbert & Pollak, 1968 [2]).

Exemple : pour 4 points en carré de côté 1, le MST donne une longueur de 3. La solution Steiner optimale atteint $1 + \sqrt{3} \approx 2,732$, soit 9 % de mieux.

1.2 Lien avec les bulles de savon

Lorsqu'on plonge un cadre métallique avec des piques (représentant les villes) dans de l'eau savonneuse, les films de savon minimisent spontanément leur surface — ce qui revient, en 2D, à minimiser la longueur totale des lignes. Ils trouvent ainsi *physiquement* les points de Steiner en respectant la règle des 120° (principe de Plateau). Ce lien entre physique, mathématiques et informatique est au cœur du sujet.

1.3 Formulation sur graphe

Entrées : graphe $G = (V, E)$, poids $\omega : E \rightarrow \mathbb{R}^+$, terminaux $T \subseteq V$.

Sortie : sous-graphe connexe H de G avec $T \subseteq V(H)$ minimisant $\sum_{e \in E(H)} \omega(e)$.

Ce problème est **NP-complet**. L'application implémente le MST (Prim, $O(n^2)$) comme approximation de facteur ≤ 2 pour $n \geq 6$.

2 Organisation et gestion de projet

2.1 Répartition du travail

Les trois membres ont contribué à **toutes les phases** du projet (algorithmes, frontend, rapport, slides). Des responsabilités principales ont structuré le travail.

Membre	Responsabilité principale	Contributions communes
AMEDIAZ Hamid	Algorithme 4 pts (16 topologies, Weiszfeld alterné); intégration backend-frontend; architecture; gestion Git.	Frontend Angular, algorithme 5 pts, rapport, slides.
BENAQA Saad	Algorithme 5 pts (~241 topologies, chaîne S0-S1-S2); scripts Python de validation; présentation orale.	Frontend Angular, algorithme 4 pts, rapport, slides.
ALAOUI KASMI Anass	Frontend Angular — canvas, controls, info-panel, header, footer; design visuel et UX; modes naïf/MST/Steiner; co-implémentation des algorithmes Weiszfeld (cas 3 pts).	Algorithmes 4 et 5 pts, backend, rapport, slides.

2.2 Rôle de l'intelligence artificielle

ChatGPT a été utilisé comme assistant de développement. Les idées, les choix technologiques (Angular, Spring Boot, Weiszfeld) et la conception algorithmique viennent entièrement de l'équipe. ChatGPT a accéléré l'implémentation de code répétitif (composants Angular, documentation) et servi de vérificateur lors du débogage. Tous les algorithmes ont été étudiés et validés manuellement avant intégration.

2.3 Méthodologie

Workflow Git structuré — une branche par fonctionnalité, pull requests avec revues croisées, fichier `ROADMAP.md` comme backlog partagé. Décisions techniques prises en sessions communes pour garantir la compréhension collective du projet.

3 Partie algorithmique

3.1 Vue d'ensemble

n points	Méthode	Type	Complexité
2	Segment direct	Exact	$O(1)$
3	Fermat-Torricelli (Weiszfeld)	Exact	$O(I)$
4	Énumération — 16 topologies	Exact	$O(16 \cdot I)$
5	Énumération — ~ 241 topologies	Exact	$O(241 \cdot I)$
≥ 6	MST — algorithme de Prim	Approx. (≤ 2)	$O(n^2)$

I = nombre d'itérations Weiszfeld (max 50 000, critère $\|\Delta F\| < 10^{-10}$).

3.2 Algorithme de Weiszfeld — 3 points

Pour trois terminaux A, B, C , le point F^* minimisant $f(F) = d(F, A) + d(F, B) + d(F, C)$ est le **point de Fermat-Torricelli**. Deux cas sont distingués :

- **Cas dégénéré** : si un angle du triangle ABC est ≥ 120 , le sommet correspondant est retourné directement (aucune itération nécessaire).
- **Cas général** : l'itération de Weiszfeld part du centroïde et converge vers F^* :

$$F^{(k+1)} = \frac{A/d_A^{(k)} + B/d_B^{(k)} + C/d_C^{(k)}}{1/d_A^{(k)} + 1/d_B^{(k)} + 1/d_C^{(k)}} \quad \text{où} \quad d_i^{(k)} = \|F^{(k)} - P_i\|. \quad (1)$$

La convergence est linéaire et garantie lorsque F ne coïncide avec aucun terminal [1]. En pratique, le critère d'arrêt $\|F^{(k+1)} - F^{(k)}\| < 10^{-10}$ est atteint en quelques centaines d'itérations pour des configurations non dégénérées.

3.3 Cas 4 points — 16 topologies

Famille	Structure	Topologies	Méthode
0 Steiner	MST (Prim)	1	Prim
1 Steiner	Fermat d'un triplet + terminal isolé	12	Weiszfeld
2 Steiner	2 paires de terminaux : $S_1 \leftrightarrow S_2$	3	Weiszfeld alterné
Total		16	

Pour la famille 2 Steiner, les deux points sont optimisés par **descente de coordonnées alternée** :

$$S_1^{(k+1)} = \text{Weber}(T_a, T_b, S_2^{(k)}), \quad (2)$$

$$S_2^{(k+1)} = \text{Weber}(T_c, T_d, S_1^{(k+1)}). \quad (3)$$

Quatre initialisations distinctes sont testées pour éviter les minima locaux.

Validation numérique : pour le carré unité $(0,0), (1,0), (1,1), (0,1)$, la longueur optimale $1 + \sqrt{3} \approx 2,732$ est retrouvée exactement par le backend Java, confirmée par le script Python indépendant.

3.4 Cas 5 points — ~ 241 topologies

La borne $k \leq n - 2 = 3$ (Gilbert & Pollak) impose au plus 3 Steiner. L'unique structure valide à $k = 3$ est la **chaîne** $S_0-S_1-S_2$.

Famille	Structure	Topologies	Méthode
0 Steiner	MST	1	Prim
1 Steiner	Fermat(triplet) + 2 terminaux raccordés	≈ 150	Weiszfeld
2 Steiner — Type I	S_0-S_1 + queue terminale	60	Weiszfeld alterné
2 Steiner — Type II	Terminal intercalé entre S_0, S_1	15	Fermat exact
3 Steiner	Chaîne $S_0-S_1-S_2$	15	Weiszfeld 3 blocs
Total		≈ 241	

L'optimisation de la chaîne se fait en **trois passes en cascade** :

$$S_0 \leftarrow \text{Weber}(T_a, T_b, S_1), \quad (4)$$

$$S_1 \leftarrow \text{Weber}(S_0, T_c, S_2), \quad (5)$$

$$S_2 \leftarrow \text{Weber}(S_1, T_d, T_e). \quad (6)$$

Plusieurs initialisations sont testées et la meilleure solution (longueur totale minimale, angles à 120 vérifiés) est retenue.

3.5 Cas $n \geq 6$ — MST

Pour $n \geq 6$, le problème de Steiner est NP-difficile. L'application retourne le **MST via l'algorithme de Prim** ($O(n^2)$), garantissant un facteur d'approximation ≤ 2 par rapport à l'optimal. Une heuristique d'insertion itérative de points de Fermat avait été développée, mais produisait des points de Steiner superposés dans certaines configurations dégénérées; elle a été remplacée par le MST pour garantir la robustesse et la stabilité visuelle.

3.6 Validation Python

Les scripts `steiner_4pts.py` et `steiner_5pts.py` (dossier `algorithms/`) utilisent `scipy.optimize.minimize` (L-BFGS-B) comme solveur indépendant du backend Java. Ils confirment les résultats sur un ensemble de cas tests couvrant configurations génériques, dégénérées et limites, validant la correction des implémentations Java.

4 Architecture de l'application

Couche	Technologie	Rôle
Frontend	Angular 19 (standalone)	Canvas interactif, modes naïf/MST/Steiner
API	REST JSON / HTTP	POST /api/steiner/solve · GET /api/steiner/health
Backend	Spring Boot 3.5 / Java 17	Algorithmes, calcul, validation des entrées
Validation	Python 3 + Scipy	Scripts de référence indépendants
Déploiement	Docker + Compose	Conteneurisation des deux services

4.1 Frontend Angular

Le frontend comprend **5 composants standalone** :

- **canvas** — rendu HTML5, placement interactif des points, affichage des arêtes et points de Steiner avec couleurs distinctives ;
- **controls** — sélection du mode de calcul, boutons lancer/réinitialiser ;
- **info-panel** — longueurs comparées des trois modes en temps réel ;
- **header et footer**.

Le service `steiner.service.ts` centralise tous les appels REST et gère la sérialisation des objets `Point`, `Edge` et `SteinerResult`.

4.2 Backend Java et API REST

Le backend s'organise en trois couches :

1. **SteinerController** — exposition REST, validation des entrées, gestion des codes HTTP 200/400/500 ;
2. **SteinerTreeService** — dispatch selon n , implémentation de tous les algorithmes ;
3. Modèles `Point` / `Edge` / `SteinerResult` — sérialisation JSON.

L'endpoint principal POST `/api/steiner/solve` reçoit une liste de points en JSON et retourne les terminaux, les points de Steiner calculés, les arêtes et la longueur totale. Un endpoint GET `/api/steiner/health` permet de vérifier la disponibilité du service.

5 Prise de recul

5.1 Bilan

L'application atteint les objectifs du sujet : solutions exactes et validées pour 2 à 5 points (propriété des 120° respectée dans tous les cas), approximation MST robuste pour $n \geq 6$, interface interactive avec comparaison simultanée des trois modes (naïf, MST, Steiner), et confirmation des résultats par scripts Python indépendants. L'architecture frontend/backend est propre, extensible et documentée.

5.2 Perspective — Extension à un grand nombre de points

La limite principale identifiée est l'absence d'un algorithme Steiner pour $n \geq 6$: seul le MST est retourné. La prochaine étape naturelle serait d'implémenter l'algorithme de **Zelikovsky** (11/6-approximation [4]), qui permettrait d'obtenir des solutions de bien meilleure qualité pour 6, 8, 10 points ou plus, tout en restant calculable en temps polynomial.

6 Références bibliographiques

Références

- [1] **Weiszfeld, E.** (1937). *Sur le point pour lequel la somme des distances de n points donnés est minimum*. Tôhoku Mathematical Journal, 43, 355–386.
- [2] **Gilbert, E. N. & Pollak, H. O.** (1968). *Steiner Minimal Trees*. SIAM Journal on Applied Mathematics, 16(1), 1–29.
- [3] **Smith, W. D.** (1992). *How to find Steiner minimal trees in Euclidean d -space*. Algorithmica, 7(1–6), 137–177.
- [4] **Zelikovsky, A.** (1993). *An $11/6$ -approximation algorithm for the network Steiner problem*. Algorithmica, 9(5), 463–470.
- [5] **Hwang, F. K., Richards, D. S. & Winter, P.** (1992). *The Steiner Tree Problem*. Annals of Discrete Mathematics, 53. Elsevier.
- [6] **Juhl, D. et al.** (2018). *The GeoSteiner software package for computing Steiner trees in the plane*. Mathematical Programming Computation, 10, 487–532.
- [7] **Druet, O.** *Vidéos pédagogiques — Arbre de Steiner et bulles de savon*. YouTube.
<https://www.youtube.com/watch?v=YsuhZ61zvLo>

Dépôt GitHub : <https://github.com/hamidamediaz/steiner-tree-solver>